RIS Lab 2 — Spring 2026 Assignment 02

Yassin Youssef, Ali Ibrahim, Yassein Mahmoud

Task 1: Teleoperation in Traffic

Task 1.1: Teleoperation

We started by connecting to the Duckietown WiFi network. We then exported the required environment variables to establish communication with our designated robot:

export ROS_MASTER_URI=http://lucky.local:11311
export ROS_IP=192.168.1.152

Then we installed the required dependency using:

sudo apt install python3-pynput

After that the virtual joystick package was cloned into the workspace and compiled:

cd ~/catkin_ws/src
git clone https://github.com/constructor-robotics/dt-gui-tools.git -b daffy
cd ~/catkin_ws
catkin_make

The interface was then launched using:

roslaunch virtual_joystick virtual_joystick_gui.launch veh:=lucky

After placing the robot on the road network, the Duckiebot was manually controlled using the graphical joystick interface. The robot was then driven to three self-determined locations while staying in the right lane to follow normal traffic rules. The navigation was done using the camera feed visualization without directly looking at the robot. The Rviz setup was similar to the one used in assignment 1, allowing us to monitor robots position and camera output.

Task 1.2: Reflection

During teleoperation, several behaviors were required to safely navigate the robot through the scene. To enable autonomous driving with similar or improved performance, the following behaviors should be implemented.

1- Line detection and intersection control:

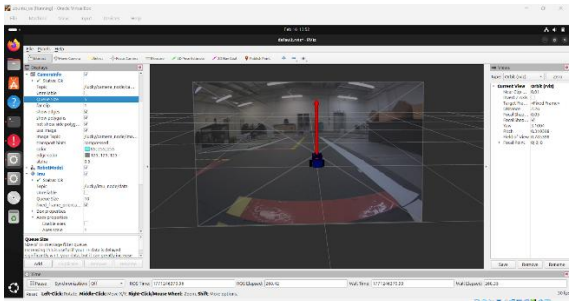

Figure 1: RViz visualization showing the camera feed and red stop line at an intersection.

While driving manually, the robot had to detect lane markings and stop at red lines before intersections. To implement this autonomously, the camera image can be used to detect red stop lines and yellow lane markings. Once a red line is detected, the robot should stop and then decide whether to turn or go straight using a simple control logic.

2- Obstacle detection:

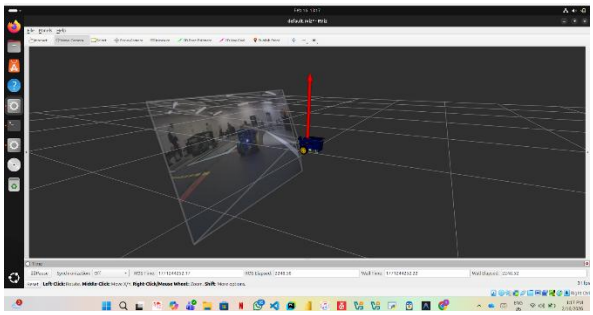

Figure 2: Rviz visualization showing the robot approaching an object.

When driving manually, we had to stop or steer to avoid hitting obstacles. Autonomously, the robot can use the frontal range sensor to measure the distance to objects. If the distance becomes too small, the robot should slow down or stop to avoid collision.

3- Drift correction:

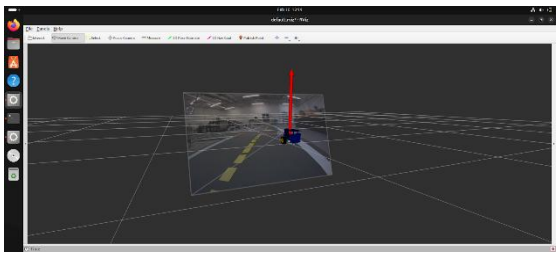

Figure 3: Rviz visualization showing the robot drifting away from the lane center.

The robot sometimes drifted away from the center of the lane, which required constant steering correction. To solve this autonomously, the robot could use its camera to detect the yellow center line and white edge line and calculate its position relative to both. When the robot starts drifting toward one side, a PID controller would adjust the steering to bring it back to the center of the lane.

Task 1.3: Introspection

To understand how the virtual joystick moves the robot's wheels, we inspected the ROS topics involved in the control pipeline using the ***rostopic info*** command.

The virtual joystick first publishes raw joystick input to the topic:

/lucky/joy

The ***joy_mapper_node*** subscribes to this topic and converts the joystick input into a ***duckietown_msgs/Twist2DStamped*** message containing linear velocity and angular velocity. This message is then published to:

/lucky/joy_mapper_node/car_cmd

The ***car_cmd_switch_node*** subscribes to this topic and forwards the command to:

/lucky/car_cmd_switch_node/cmd

Next, the **kinematics_node** subscribes to this command and converts the linear and angular velocity into individual wheel velocities. It publishes a ***duckietown_msgs/WheelsCmdStamped*** message to:

/lucky/wheels_driver_node/wheels_cmd

Finally, the ***wheels_driver_node*** sends electrical signals to the motors, which causes the wheels to rotate.

The overall message flow is:

Joystick GUI, joy_mapper_node, car_cmd_switch_node, kinematics_node, wheels_driver_node, motors

This confirms how joystick input is transformed step-by-step into physical wheel movement.

Task 2: Calibration

Task 2.1: Software Preparation

In the previous assignment, we created a node that directly published ***duckietown_msgs/WheelsCmdStamped*** messages to control the wheels.

For this task, the node was modified to publish ***duckietown_msgs/Twist2DStamped*** messages instead. The message is now published on the first topic in the teleoperation pipeline:

/lucky/joy_mapper_node/car_cmd

The Twist2DStamped message contains two main fields: v(linear velocity) and omega(angular velocity).

By publishing this message, the command is processed through the normal control pipeline:

joy_mapper_node, car_cmd_switch_node, kinematics_node, wheels_driver_node

This ensures that the velocity commands are converted properly into individual wheel speeds by the kinematics node.

After modifying the node and running it, the robot successfully moved forward using the new message type, confirming correct integration with the control pipeline.

Task 2.2: Calibration

To calibrate the trim parameter, a straight section of the road along four mats was selected. The robot was placed and was aligned with the road.

For each trial, a trim value was set using:

rosparam set /lucky/kinematics_node/trim <value>

Then we ran our node to drive the robot forward. After each run, we measured the perpendicular distance between the robot's final position and the lane center.

The following trim values and drift distances were recorded:

Trim Value	Drift (cm)
-0.02	0.5
-0.01	21
0.0	78
0.005	46
0.01	29
0.012	49
0.02	7

We ran a total of seven trials.

The trim value -0.02 produced the smallest drift of only 0.5 cm, meaning the robot stayed almost perfectly centered in the lane. We selected -0.02 as our final trim value.

Task 2.3: Reporting and Evaluation

Using matplotlib*, we created a scatter plot with trim values on the x-axis and the measured drift distance on the y-axis.*

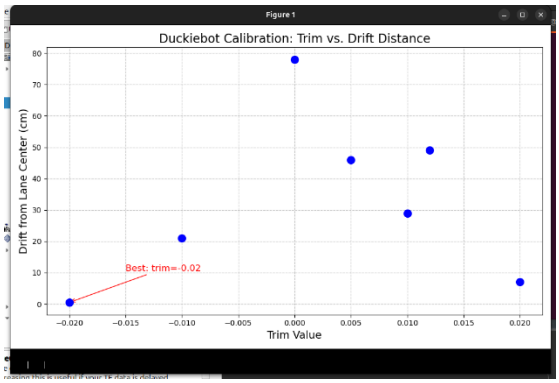

Figure 4: Scatter plot of trim values versus drift distance.

The plot clearly shows that trim = -0.02 resulted in the smallest drift of only 0.5 cm. As the trim value moves further away from -0.02 in either direction, the drift increases significantly. The worst result occurred at trim = 0.0, where the drift reached 78 cm.

Based on the plot, we selected -0.02 as our final trim value. To permanently save this calibration on the robot, we used:

rosparam set /lucky/kinematics_node/trim -0.02
rosservice call /lucky/kinematics_node/save_calibration

The plotting script used to generate the scatter plot is included in the code submission.

Task 2.4: Evaluating Stability

After selecting the best trim value (-0.02), the experiment was repeated six times. The measured drift distances were: 17 cm, 20 cm, 7 cm, 5 cm, 15 cm, and 16 cm.

Mean = 13.3 cm

Standard Deviation ≈ 5.8 cm

The relatively high variability shows that the robot's behavior is not fully deterministic. Physical factors such as floor surface conditions, wheel slippage, a loose right wheel, small differences in starting alignment, and other minor effects all influence the final position.

We would not adjust the trim value again, since the variation is mainly caused by real-world physical and mechanical factors rather than the trim parameter itself. No single trim value can completely eliminate this type of noise.

Task 3: Revisiting old tasks

We implemented a ROS node that drives the Duckiebot along the path of a “Haus des Nikolaus” on the floor. The node publishes ***duckietown_msgs/Twist2DStamped*** messages to:

/lucky/joy_mapper_node/car_cmd

The path is generated as a sequence of straight driving segments and in-place rotations. The robot traces the house shape in the following order:

- bottom left to top left to top right to bottom right to bottom left (square)
- bottom left to top right (diagonal)
- top right to roof peak to top left (roof)
- top left to bottom right (final diagonal)

Each movement segment is defined by a forward speed, a turning speed, and a duration. Between segments, the robot stops briefly to stabilize before starting the next motion.

Getting the shape correct required trial and error. The rotation durations had to be tuned manually because the robot did not always rotate the same amount for the same command. Similarly, the forward durations had to be adjusted until the side lengths looked roughly consistent.

During testing, the robot consistently drifted to the right due to a loose right wheel, which caused straight segments to curve slightly. The square part of the house worked reasonably well, but the later parts, especially the diagonals and roof, were less accurate, since small errors from earlier segments accumulated and affected the following segments. We also observed occasional wireless lag between the laptop and the robot, which sometimes caused commands to be delayed or cut short.

Comparison Between Turtlesim (Lab 1) and the Real Duckiebot:

In Lab 1, the Turtlesim house was written in C++ and used closed-loop feedback control. The node subscribed to /turtle1/pose, which provided the turtle's exact x and y position and heading at 60 Hz. Because of this constant feedback, the node was able to use a rotateTo function that turned the turtle precisely to a target angle using proportional control, and a driveStraightTo function that drove to exact coordinates while continuously correcting its heading.

The house was defined as a set of waypoints using a struct such as A{3.0, 3.0} with exact coordinates, and the turtle simply moved between these points. As a result, the house was perfectly shaped, with sharp corners and straight edges every single time it was executed.

On the real Duckiebot, this level of precision was not possible. There is no pose topic that provides the robot's position on the floor, so we could not use feedback control. Instead, our Python node relied on open-loop control, where each movement was defined as a (speed, omega, duration) tuple. For example, (0.3, 0.0, 2.0) means "drive forward at speed 0.3 for 2 seconds." This meant we had no way of specifying exact target positions, only approximate movement durations.

In the simulation, the rotateTo function kept adjusting until the heading error was below 0.01 radians, ensuring accurate turns. On the real robot, a turn command with the same duration sometimes resulted in 85° and other times 95°, with no way to correct the difference. Similarly, in Turtlesim, the driveStraightTo function reduced forward motion if the heading drifted too much, keeping lines perfectly straight. On the real robot, however, the loose right wheel caused every straight segment to curve slightly, and without feedback, there was no correction possible.

The most noticeable difference was error accumulation. In simulation, each segment began exactly at the intended target position because the feedback loop ensured accuracy. On the real robot, each segment started from wherever the previous one ended, meaning small errors built up over time. By the final diagonal, the shape was clearly distorted compared to the clean result in Turtlesim.

Overall, this comparison shows that simulation provides an ideal and controlled environment where commands execute perfectly. In contrast, real world robotics involves hardware imperfections, mechanical inconsistencies, and environmental factors that make open-loop control much less reliable without feedback.

Contributions:

Yassin Youssef = Task2, Task3

Yassein Mahmoud = Task1

Ali Ibrahim = lab report